IF2110 Algoritma dan Pemrograman 2

Ekspresi Kondisional

Tim Pengajar IF2110

Sekolah Teknik Elektro dan Informatika

Apa yang sudah dipelajari sebelumnya?

- Abstraksi persoalan dengan menggunakan pendekatan berpikir fungsional
- Membuat program kecil dalam notasi fungsional dan notasi Haskell yang melibatkan ekspresi aritmatika dan logika

Ekspresi Kondisional

- Ekspresi kondisional:
  - ekspresi yang keluarannya tergantung kepada hasil evaluasi beberapa kondisi
  - hasil analisis kasus (dekomposisi persoalan, pemecahan secara independen)
- Setiap kasus harus **disjoint** dan analisa kasus harus **mencakup semua kasus**

Ekspresi Kondisional (cont.) Notasi dan Evaluasi

- Jenis-jenis notasi kondisional:
  - Notasi **if-then-else** untuk 2 kasus saling komplementer
  - Notasi **depend on**
  - Notasi **depend on dengan else** (otherwise)
- Tidak ada notasi if-then karena semua kasus harus ada nilainya → fungsi harus mengembalikan nilai, termasuk ketika kondisi dalam "if" tidak terpenuhi
- Urutan kondisi (pada depend on) tidak penting (komutatif)
- Evaluasi ekspresi parsial (hanya untuk kondisi yang evaluasinya true)

Dua Kasus Saling Komplementer

- Persoalan mencari nilai maksimum dari dua buah nilai integer, misal: a dan b
  - Kasus 1: $a \geq b$
  - Kasus 2: $a < b$
- Kasus 1 dan 2 disjoint.
- Tidak ada kasus lain selain 2 kasus tsb.

Dua Kasus Saling Komplementer (cont.)

```
<Nama-fungsi> (<list parameters>) :
      if  <kondisi-1>  then
            <ekspresi-1>
      else   { not kondisi-1 }
            <ekspresi-2>
```

REALISASI

```
max2(a,b) : if (a>=b) then
                a
            else { a<b }
                b
```

APLIKASI

⇒ max2(-1,5)

Analisis Kasus: Lebih Dari 2 Kasus

- Persoalan mencari nilai maksimum dari tiga buah nilai integer yang tidak sama, misal: a, b, c
- Kasus 1 : a > b and a > c
- Kasus 2 : b > a and b > c
- Kasus 3 : c > a and c > b
- Kasus 1, 2, 3 disjoint dan lengkap

Analisis Kasus: Lebih Dari 2 Kasus (cont.)

```
<Nama-fungsi> (<list parameters>) :
   depend on (<list variables>)
      <Kondisi-1> : <Ekspresi-1>
      <Kondisi-2> : <Ekspresi-2>
      <Kondisi-3> : <Ekspresi-3>
   [else          : <Ekspresi-4>]
```

REALISASI

```
max3(a,b,c) :
   depend on (a, b, c)
      (a>b) and (a>c) : a
      (b>a) and (b>c) : b
      (c>a) and (c>b) : c
```

APLIKASI

⇒ max3(-1,5,3)

Studi Kasus: Maksimum 3 bilangan

- Buatlah definisi, spesifikasi dan realisasi fungsi max3 yang menerima 3 buah bilangan bulat yang nilainya tidak sama dan menghasilkan maksimum dari ke-3 bilangan tsb.
  - Versi 1: identifikasi domain, berangkat dari hasil (solusi pada slide sebelumnya)
  - Versi 2: berdasarkan letak ketiga bilangan pada sumbu bilangan (terdapat 6 kemungkinan kasus)
  - Versi 3: reduksi domain (bandingkan 2 nilai, lalu bandingkan hasilnya dgn nilai ke-3)
  - Versi 4: memanfaatkan fungsi max2, aplikasikan max2 sebanyak 2 kali

Maksimum 3 Bilangan (versi 2)

Berdasarkan letak ketiga bilangan pada sumbu bilangan.
Ada enam kemungkinan letak ke tiga nilai tersebut pada sumbu bilangan.

```
MAKSIMUM 3 BILANGAN (versi 2)                    max3(a,b,c)
DEFINISI DAN SPESIFIKASI
max3 : 3 integer → integer
  { max3(a,b,c) menentukan maksimum dari 3 bilangan integer yang
    berlainan nilainya, a ≠ b dan a ≠ c dan b ≠ c }
-- REALISASI
max3 (a,b,c) :
   depend on (a, b, c)
      a > b and b > c : a
      a > c and c > b : a
      b > a and a > c : b
      b > c and c > a : b
      c > b and b > a : c
      c > a and a > b : c
```

Maksimum 3 Bilangan (versi 3)

Reduksi domain fungsi: Ambil 2 dari 3 nilai, bandingkan. Manfaatkan hasil perbandingan untuk menentukan maksimum dengan cara membandingkan terhadap bilangan ketiga.

```
MAKSIMUM 3 BILANGAN (versi 3)                          max3(a,b,c)
DEFINISI DAN SPESIFIKASI
max3 : 3 integer → integer
  { max3 a b c menentukan maksimum dari 3 bilangan integer yang
    berlainan nilainya, a ≠ b dan a ≠ c dan b ≠ c }
REALISASI
max3(a,b,c) : if (a > b) then
                  if (a > c) then a
                  else c
              else -- a < b
                  if (b > c) then b
                  else c
```

AND THEN dan OR ELSE

- Operator AND THEN
  - Ekspresi **A AND then B**: B hanya dievaluasi jika Ekspresi A bernilai true.
- OPERATOR OR ELSE
  - Ekspresi **A OR else B**: B hanya dievaluasi jika Ekspresi A bernilai false.

Ekpresi boolean	Ekivalen dengan
A **and then** B	**if** A **then** B **else** **false**
A **or else** B	**if** A **then** **true** **else** B

Latihan 1

- Buatlah realisasi dari fungsi di bawah ini berdasarkan definisi dan spesifikasi yang diberikan. Jika perlu membuat fungsi antara, buatlah definisi, spesifikasi dan realisasinya.

```
APAKAH DATE VALID?                                  isDateValid(d,m,y)
---------------------------------------------------------------------
DEFINISI DAN SPESIFIKASI
isDateValid :  3 integer → boolean
 { isDateValid(d,m,y) mengembalikan nilai true jika
   d, m, y membentuk date yang valid. Definisi date yang
   valid adalah jika elemen hari (d) bernilai antara 1
   dan 31, tergantung pada bulan dan apakah tahun kabisat
   atau bukan, elemen bulan (m) bernilai antara 1 dan 12,
   dan elemen tahun (y) bernilai antara 0 dan 99.
   Nilai y mewakili tahun 1900 s.d. 1999 }
```

Latihan 2

- Buatlah definisi, spesifikasi, dan realisasi dari fungsi **nilaiTengah** yang menerima masukan 3 buah integer yang berlainan nilainya yang urutannya bisa acak dan mengembalikan sebuah integer yang merupakan salah satu dari ke-3 nilai tersebut yang jika diurutkan berada di tengah.
- Contoh aplikasi fungsi:

```
⇒ nilaiTengah(1,2,3)
2
⇒ nilaiTengah(-6,1,5)
1
⇒ nilaiTengah(-1,-4,10)
-1
```

- Petunjuk: Buatlah fungsi min3 dan max3 dalam realisasinya.

Translasi ke Haskell

Dua Kasus Saling Komplementer if-then-else

```
<Nama-fungsi> <list parameters> =
      if  <kondisi-1>  then
                  <ekspresi-1>
      else   -- not kondisi-1
                  <ekspresi-2>
```

```
-- REALISASI
max2 a b = if (a>=b) then
                a
            else -- a<b
                b
```

```
-- APLIKASI
-- max2 (-1) 5
```

Analisis Kasus: Lebih dari Dua Kasus depend-on

- Pada Haskell, notasi "depend on" menggunakan **guard**/tanda *pipe* (|)
- Sintaks:

```
<Nama-fungsi> (<list parameters>)
    |  <kondisi-1> = <ekspresi-1>
  [ |  otherwise   = <ekspresi-2> ]
```

```
max3 a b c
  | (a>b) && (a>c) = a
  | (b>a) && (b>c) = b
  | (c>a) && (c>b) = c
```

Dua Kasus Saling Komplementer depend-on dengan otherwise

```
max2 a b
   | (a >= b)  = a
   | otherwise = b  -- a<b
```

```
-- Aplikasi
-- max2 7 (-5)
```

Analisis Kasus: Lebih Dari 2 Kasus if-then-else

```
max3 a b c =
    if (a>b) && (a>c) then a
    else if (b>a) && (b>c) then b
    else c
```

Bahan

- Diktat “Dasar Pemrograman, Bag. Pemrograman Fungsional” oleh Inggriani Liem, revisi Februari 2014
- “Haskell The Craft of Functional Programming”; Simon Thompson; Second Edition; Addison-Wesley
- “Real World Haskell”; B. O’Sullivan, J. Goerzen, D. Stewart; O’Reilly